Geographic Address Assignment Optimization for Emergency Services Workers

Bob Iannucci
Iannucci Wireless Systems Engineering
Palo Alto, CA 94306
bob@rail.com

***Abstract*—During disaster response, Emergency Services Workers (ESWs) must visit assigned addresses to assess damage. We study the problem of partitioning addresses across N workers to minimize assessment completion time. We model the street network as road segments between intersections, each with a precomputed traversal cost matrix, and constrain all walking distances to follow streets. Three metrics capture what matters operationally: *walk quality* (fraction of walking that visits addresses vs. transitions between segments), *time spread* (difference between the first and last ESW to finish), and the *productive-distance invariant* (the observation that productive walking is determined by the address set, not the algorithm — only unproductive walking can be optimized). We evaluate eleven algorithms — nearest-neighbor chain, right-hand rule, 2-opt local search, recursive bisection, Voronoi with balanced seeds, Voronoi with boundary transfer, BFS, DFS, simulated annealing, a random baseline, and an oracle — across five Palo Alto neighborhoods chosen for maximum geometric diversity, at team sizes $N = 3, 5, 7, 10$. No single algorithm dominates: DFS and 2-opt achieve the highest walk quality on compact neighborhoods (66%), while Voronoi with boundary transfer wins on elongated layouts (59%). All algorithms fail on sparse campuses (<26% walk quality). The oracle reduces worst-case walking by 25% vs. random assignment.**

***Index Terms*—disaster response, geographic partitioning, vehicle routing, emergency services, road network, address assignment**

I. Introduction

Post-disaster damage assessment requires Emergency Services Workers (ESWs) to physically visit assigned addresses, inspect structures, and report conditions. The speed of this process directly affects emergency response effectiveness [1]. In the ES-Link system [2], a Neighborhood Preparedness Coordinator (NPC) assigns addresses to ESWs through a web-based map interface. Manual assignment is tedious and does not optimize for worker travel distance.

We study the automatic assignment problem: given a set of addresses on a road network and N workers, find a partition that minimizes the time until all assessments are complete. The problem combines elements of the Capacitated Vehicle Routing Problem (CVRP) [3] and balanced graph partitioning [4], but differs in that our "vehicles" are on foot, start from a shared staging area, and must walk on streets — they cannot jump between non-adjacent addresses. The general problem is NP-hard [5], so we seek practical heuristics that produce good solutions in real-time.

Copyright © 2026 by Bob Iannucci. All rights reserved worldwide.

Through development and evaluation of eleven algorithms, we discover three insights that are not obvious from the problem statement:

1) **Walk quality matters more than distance or time.** ESWs reject assignments with excessive purposeless walking regardless of what the time metric shows.
2) **Productive walking is invariant.** The distance spent visiting addresses is determined by the address set, not the algorithm. Only *unproductive* walking (transitions between segments) can be optimized.
3) **Neighborhood geometry determines which algorithm wins.** No single method dominates across dense, grid, cul-de-sac, sparse, and elongated layouts.

A. Contributions

C1. A road-network segment model with a geometric pipeline for address snapping and side-of-street classification (Section II).

C2. A per-segment cost matrix that captures all traversal options as an emergent property of address geometry (Section II-B).

C3. Three walking metrics — walk quality, time spread, and the productive-distance invariant — that capture what matters operationally (Section III).

C4. Empirical evaluation across five diverse neighborhoods at $N = 3, 5, 7, 10$ showing the winning strategy depends on neighborhood geometry (Section VII).

C5. An oracle algorithm that runs all strategies and selects the best, reducing worst-case walk by 25% vs. random assignment, in under one second (Section V-J).

C6. A time-based reformulation showing that walking quality must be a hard constraint, not just a cost component (Section VII-G).

C7. Voronoi partitioning with balanced seeds and boundary transfer wins decisively on elongated neighborhoods (59% walk quality on Southgate vs. 28% for chain methods), overturning the naive Lloyd's negative result (Section VII-H).

C8. A constrained formulation revealing that walkable contiguity, time balance, and walking quality are in fundamental tension — the resolution depends on operational context (Section VII-I).

II. Problem Formulation

A. Road Network Model (C1)

We model the street network as a graph $G = (V, E)$ extracted from OpenStreetMap (OSM), where vertices V are *intersections* and edges E are *segments*.

Intersection: A node shared by two or more OSM ways, or a way endpoint. We identify intersections by counting way membership: any node appearing in more than one way, plus all way endpoints, is an intersection.

Segment: A maximal chain of road edges between two intersection nodes, with street name, road class, polyline geometry, and length D (meters). Within a segment, no turns are possible; turns occur only at endpoints. Each segment endpoint has two *ports* (one per sidewalk): S_L, S_R at the start intersection and E_L, E_R at the end.

1) Address Snapping: Each address has a geographic coordinate (latitude, longitude) from the municipal database. We snap it to the road network by finding the nearest point on any segment polyline:

1) For each edge (v_i, v_{i+1}) of each segment's polyline, compute the point-to-line-segment distance using perpendicular projection, clamped to $[0, 1]$.
2) Select the segment and edge with minimum distance. Reject addresses $> 200\,\text{m}$ from any segment (only 3 of 41,857 addresses).
3) Compute the fractional position $t \in [0, 1]$ along the segment by accumulating edge lengths up to the projection point.

2) Side-of-Street Classification: Each snapped address is classified as *left* or *right* of the segment centerline using the cross product of the road direction vector $\vec{d} = (v_{i+1} - v_i)$ and the address displacement vector $\vec{a} = (p - v_i)$:

$$\text{side} = \begin{cases} \text{left} & \text{if } \vec{d} \times \vec{a} \geq 0 \\ \text{right} & \text{otherwise} \end{cases} \tag{1}$$

This geometric classification is more reliable than house-number parity (even/odd), which can be inconsistent across jurisdictions.

3) Data Quality: From 41,857 municipal addresses within the Palo Alto bounding box, 41,854 snap successfully to one of 7,430 address-bearing segments. Snap distances have median 22.9 m (consistent with the typical setback of houses from the road centerline), 87% are within 30 m, and only 1% exceed 50 m. Segment lengths have median 63 m, mean 83 m, and range from 0.5 m to 2,288 m. Addresses per segment: 31% have one address, 39% have 2–5, 15% have 6–10, and 15% have 11 or more (up to 135).

B. Segment Walking Distance (C2)

Given a segment with addresses, how should an ESW walk it? The answer depends on *which end* they enter from, *which end* they exit, and *which side* of the street they are on at each end. Since segments are stitched together into ESW assignments, the entry and exit are degrees of freedom that the assignment algorithm must choose. We need a representation that captures all options.

1) Definitions: Let D be the segment length, w the street width (estimated from OSM road class: residential $\approx 10\,\text{m}$, service $\approx 6\,\text{m}$, secondary $\approx 14\,\text{m}$), and n the address count. Each segment endpoint has two *ports* (one per sidewalk): S_L, S_R at the start intersection and E_L, E_R at the end, giving four ports total. Address positions are expressed as fractional distance $t \in [0, 1]$ along the segment. Each address is classified as left or right of the centerline (Eq. 1). A *run* is a maximal consecutive group of addresses on the same side when sorted by t; the number of runs r determines how many street crossings are needed. The *mean same-side inter-address gap* $\bar{g}$ is the average distance between consecutive addresses on the same side of the street; it governs the choice of traversal strategy (Section II-B3). A *sub-segment* is one side of a segment — the left-side addresses or the right-side addresses. A through-segment has two sub-segments; a cul-de-sac has one, because the ESW must traverse both sides in a single out-and-back and the two sides cannot be independently assigned. We consider the choice of assignment granularity — segments vs. sub-segments — in Section II-B4.

Traversal strategies. Two strategies cover all cases:

Through-traversal (enter at one end, exit at the other). The ESW walks monotonically along the segment, crossing the street at each run boundary:

$$C(S_s, E_e) = D + k(s, e) \cdot w \tag{2}$$

where $k(s, e) = (r-1) + [s \neq s_1] + [e \neq s_r]$ counts street crossings: $(r-1)$ mandatory crossings at run boundaries, plus one if the entry side s differs from the first run's side s_1, plus one if the exit side e differs from the last run's side s_r. The notation $[\cdot]$ is the Iverson bracket.

Out-and-back (enter and exit at the same end). The ESW walks to the farthest address and returns:

$$C(S_s, S_e) = 2 \cdot t_n \cdot D + k_{\text{out}}(s) \cdot w \tag{3}$$

where t_n is the fractional position of the farthest address.

Cul-de-sac constraint. A segment with a dead end (one endpoint has degree 1 in the road graph) can only be entered and exited from its open end. Through-traversal is physically impossible — the ESW would walk into a dead end with no connecting street. The cost matrix still has 16 entries, but the assignment algorithm must only use the out-and-back entries from the open end.

2) Why Not Solve TSP per Segment?: The within-segment problem — visiting n addresses on two sides of a street — is a special case of the TSP. Solving it exactly on general points would be intractable for our largest segment (135 addresses: 135! permutations). However, the 1D structure of a street reduces the problem to a choice between two monotonic strategies — neither requires backtracking within a pass:

Through-traversal (zigzag): The ESW makes a single pass from one end to the other, crossing the street at each run boundary. Cost: $D + (r-1) \cdot w$ (segment length plus one crossing per run boundary).

Out-and-back (U-turn): The ESW sweeps one side on the way out and the other side on the way back. Cost: $\approx 2 \cdot t_n \cdot D$ (twice the distance to the farthest address, no crossings).

The trade-off is direct: through-traversal walks the segment once but pays $(r-1) \cdot w$ in crossings; out-and-back avoids crossings but pays $\approx 2 \cdot t_n \cdot D$ to walk the segment twice. Through-traversal wins when:

$$(r-1) \cdot w < (2t_n - 1) \cdot D \tag{4}$$

The number of runs r is directly related to the mean same-side inter-address gap $\bar{g}$: when $\bar{g} < w$, addresses alternate sides frequently, producing many runs and a large crossing penalty $(r-1) \cdot w$ that eventually exceeds the backtrack cost. When $\bar{g} > w$, there are few runs and the crossing penalty is small.

The cost of each strategy depends only on which side the ESW starts and finishes — precisely the four ports. The cost matrix $\mathbf{C}$ is a 4×4 matrix encoding the minimum walking distance for every (entry port, exit port) combination. It evaluates both strategies for all 16 port pairings and records the cheaper result. The matrix is precomputed in $O(n)$ time from the sorted address sequence — even for the 135-address worst case, this is instantaneous.

3) Empirical Strategy Selection: In our dataset of 3,222 segments with addresses on both sides, $\bar{g}/w$ cleanly separates the two outcomes:

- **88% prefer through-traversal** (2,823 segments). Median $\bar{g}/w = 1.9$: same-side addresses are nearly two street-widths apart, producing few runs and a small crossing penalty.
- **12% prefer out-and-back** (399 segments). Median $\bar{g}/w = 0.5$: same-side addresses are packed at half the street width, producing many runs whose accumulated crossing cost exceeds the price of walking the segment twice.

The $\bar{g}/w$ ratio predicts the cost matrix outcome with 79% accuracy. The remaining 21% are boundary cases where the exact address positions and port choices matter — the cost matrix handles these correctly.

4) Segments vs. Sub-Segments: Should the assignment algorithm assign whole segments (both sides of the street) to one ESW, or could it assign each sub-segment (one side) independently?

Sub-segment assignment is the stronger formulation in general. If ESW 1's route passes through a segment from S to E and ESW 2's route passes through the same segment from E to S, assigning one sub-segment to each is a productive use of both workers' walking — each visits addresses on their side while traversing the segment in opposite directions.

However, sub-segment decomposition only helps when the whole-segment cost matrix would have selected out-and-back (U-turn) — that is, when $(r-1) \cdot w \geq D$ (Eq. 4). In that case, a single ESW already walks ∼2D to visit both sides; splitting into sub-segments lets two ESWs share this cost at ∼D each. When zigzag is optimal ($(r-1) \cdot w < D$), one ESW walks $D + (r-1)w < 2D$ — strictly less than two ESWs walking D each. Decomposition would *increase* total walking.

In our dataset, $(r-1) \cdot w < D$ for *every* both-side through-segment across all five neighborhoods. Zigzag universally wins: residential street geometry (median segment length 63 m, typical $r = 3$–5, $w = 10$ m) never produces enough crossings to exceed the segment length. Cul-de-sacs cannot be decomposed regardless (both sides must be traversed together).

We therefore use **whole segments with 4×4 cost matrices** as the atomic unit. Sub-segment assignment remains a valid optimization for street geometries with denser address alternation ($\bar{g} < w$), but it does not arise in the Palo Alto data.

5) Worked Example: We compute the cost matrix for two representative Fairmeadow segments: a through-segment (Starr King Circle, both ends connected) and a cul-de-sac (Wright Place, dead end at the start node).

Starr King Circle (Fig. 1): $D = 243$ m, 19 addresses (8 left, 11 right), $r = 14$ runs, $w = 10$ m. Both endpoints connect to other streets (degree 3), so all four traversal directions are reachable.

Starr King Circle: 19 Addresses, 14 Runs

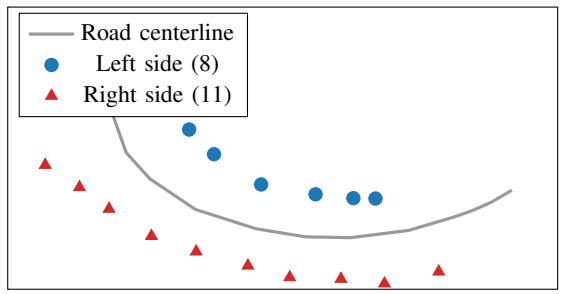


Fig. 1. Starr King Circle (243 m, 19 addresses). Blue circles = left-side addresses (8). Red triangles = right-side addresses (11). Black line = road centerline. Both endpoints connect to the road network, so through-traversal and out-and-back are both feasible. The 14 runs produce 13 mandatory crossings.

The through-traversal cost is $C(S_R, E_L) = 243 + 13 \times 10 = 373$ m (the cheapest entry: start on the right to match the first run's side, exit left to match the last). The out-and-back alternative $C(E_R, E_R) = 2 \times 243 + 10 = 531$ m — 42% more. With $\bar{g}/w = 1.1$, the cost matrix correctly selects zigzag. The full matrix (meters):

	S_L	S_R	E_L	E_R
S_L	569	569	383	393
S_R	559	559	**373**	383
E_L	383	393	531	531
E_R	**373**	383	541	541

Wright Place (cul-de-sac): $D = 76$ m, 7 addresses (4 left, 3 right), $r = 6$ runs, $w = 10$ m. The start node is a dead end (degree 1); the end node connects to the network (degree 4). An ESW arriving from the road network *must* enter at E and *must* exit at E — through-traversal would strand them at the dead end. The relevant matrix entries are the out-and-back cells $E \to E$:

	S_L	S_R	E_L	E_R
S_L	160	160	136	126
S_R	170	170	146	136
E_L	136	126	212	212
E_R	146	136	**202**	**202**

The cheapest reachable option is $E_R \to E_L = 202\,\text{m}$ (or $E_R \to E_R = 202\,\text{m}$): enter on the right, walk to the dead end visiting addresses, return. The matrix also contains through-traversal entries ($E \to S$: 126–146 m) that are geometrically cheaper but *unreachable* — the dead end has no connecting street. The assignment algorithm enforces this by checking node degree when selecting ports.

Fig. 2 shows the extreme case of Roosevelt Circle, where the U-turn strategy saves 81% over zigzag on a short, densely addressed segment ($\bar{g}/w \ll 1$).

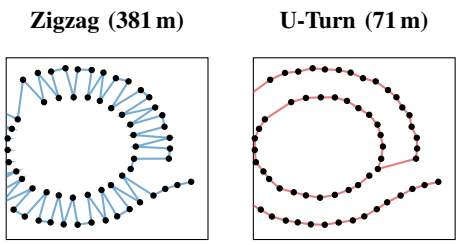


Fig. 2. Roosevelt Circle (71 addresses, $\bar{g}/w \ll 1$). Left: zigzag crosses at every run boundary (381 m). Right: U-turn sweeps each side (71 m), an 81% reduction. The cost matrix selects the optimal strategy automatically for every segment.

C. Walking Distance

The total walking distance for ESW k, assigned an ordered sequence of segments $P_k = (seg_1, seg_2, \ldots, seg_m)$ with chosen ports, is:

$$W_k = \underbrace{\sum_{i=1}^{m} C(\text{in}_i, \text{out}_i)}_{W_k^{\text{prod}}} + \underbrace{\sum_{i=1}^{m-1} d_{\text{road}}(\text{out}_i, \text{in}_{i+1})}_{W_k^{\text{unprod}}} \tag{5}$$

- $C(\text{in}_i, \text{out}_i)$ is the cost of traversing segment i, read from its precomputed cost matrix. This is **productive walking**: every meter spent visiting addresses.
- $d_{\text{road}}(\text{out}_i, \text{in}_{i+1})$ is the shortest-path distance along the road network from the exit port of segment i to the entry port of segment $i{+}1$, computed by Dijkstra's algorithm on the intersection graph. This is **unproductive walking**: meters spent traveling between segments without visiting any addresses.

Theoretical ideal. If addresses could be partitioned with zero unproductive walking and perfect balance:

$$W^* = \frac{1}{N} \sum_{\text{all segments}} \min_{\text{in,out}} C(\text{in}, \text{out}) \tag{6}$$

This is unachievable because: (1) geographic partitioning creates boundaries requiring inter-segment transitions, and (2) integer segment counts prevent exact balance. The **overhead ratio** $\rho = W_{\max}/W^*$ measures how close the assignment comes. $\rho = 1.0$ is perfect; we observe ρ between 1.6 and 2.8.

Objective. Minimize the worst-case walking distance:

$$\min_{\{P_1,\ldots,P_N\}} \max_k W_k \qquad \text{s.t.} \quad \frac{\max_k |P_k|}{\min_k |P_k|} \le \beta \tag{7}$$

where $|P_k|$ is the address count in partition k and $\beta \approx 1.0$ is the balance tolerance.

D. General Constraints

The following constraints apply to all algorithms and evaluations:

Whole-segment assignment. Each segment is assigned to exactly one ESW. No segment is split between workers. This is a hard constraint: an ESW visiting half a street while another visits the other half is operationally confusing and creates boundary disputes.

Shared starting location (huddle point). All ESWs begin at a shared staging area — the neighborhood intersection nearest to the serving fire station. The walk from huddle to each ESW's first segment is included in their total walking distance and time.

Street-constrained walking. ESWs must follow the road network at all times. They may walk along a segment, turn at an intersection, or cross a street, but may not jump over houses or cut through properties. All inter-segment distances use Dijkstra shortest paths on the road network.

Assessment time. Each address requires a fixed assessment time $t_{\text{assess}} = 5\,\text{min}$. The finish time for ESW k is:

$$T_k = \frac{W_k}{v} + |P_k| \cdot t_{\text{assess}} \tag{8}$$

where $v = 5\,\text{km/h}$ is walking speed and $|P_k|$ is the address count. Assessment time typically dominates walking time by $\sim 8\times$.

E. Data

Segments are extracted from OpenStreetMap via `osm2pgsql` (Geofabrik California PBF, 1.3 GB) filtered to the Palo Alto bounding box (37.35–37.48°N, 122.05–122.20°W). From 35,912 OSM road ways and 146,957 nodes, we identify 54,854 intersections and build 64,482 segments. Of these, 7,430 have at least one snapped address. Road-network shortest paths are precomputed by Dijkstra within each neighborhood (2–31 s depending on size). Huddle points use 6 Palo Alto Fire Department stations, assigned to neighborhoods by centroid proximity.

III. Three Metrics

A. Walk Quality

Walk quality is the fraction of an ESW's total walking that is productive:

$$Q_k = \frac{W_k^{\text{prod}}}{W_k^{\text{prod}} + W_k^{\text{unprod}}} \tag{9}$$

Why it matters to individuals. An ESW who walks past unvisited addresses to reach their assigned territory perceives the assignment as incompetent. Walk quality below 50% means the ESW spends more time transitioning than assessing — operationally unacceptable regardless of what the time metric shows.

Why it matters to the response. If ESWs lose confidence in their assignments, they may deviate from the prescribed route, creating coverage gaps. Walk quality is a proxy for assignment credibility.

B. Time Spread

The time spread is the difference between the first and last ESW to finish:

$$\text{Spread} = \max_k T_k - \min_k T_k, \qquad T_k = \frac{W_k}{v} + |P_k| \cdot t_{\text{assess}} \quad (10)$$

where $v = 5\,\text{km/h}$ is walking speed, $|P_k|$ is the address count, and $t_{\text{assess}} = 5\,\text{min}$ is the per-address assessment time.

Why it matters to individuals. Fairness — no ESW should work hours longer than another for the same event.

Why it matters to the response. The NPC cannot act on partial data. The straggler determines when relief is allocated. A 100-minute spread means the NPC waits 100 minutes after the fastest ESW finishes before having complete damage data.

C. Productive-Distance Invariant

The total productive walking distance $\sum_k W_k^{\text{prod}}$ is determined by the address set and the segment cost matrices — it is *invariant across algorithms*. All algorithms visit the same segments; the only difference is *which order* and *which ESW*. This means:

$$\sum_k W_k^{\text{prod}} \approx \text{constant for all algorithms} \quad (11)$$

Significance. Algorithms can only reduce unproductive walking, not productive walking. This reframes the optimization: instead of "minimize total distance," the objective is "minimize waste." Every comparison confirms this: the productive column in Table II is nearly constant within each neighborhood; the unproductive column tells the entire story.

IV. The Neighborhoods

We select five Palo Alto neighborhoods spanning the widest diversity of geometry, density, and street pattern. This diversity ensures that our algorithm comparisons expose real performance differences rather than artifacts of a single layout.

Fig. 3 shows OpenStreetMap views of each neighborhood, illustrating the dramatic differences in street geometry.

Palo Alto Central (PAC). 145 addresses packed into 11 segments at 9,686 addresses/km^2. Nearly all segments are adjacent — transitions are short. The challenge is partitioning a single dense cluster.

Fairmeadow (FM). 307 addresses across 43 segments in a cul-de-sac layout. Streets radiate from a central spine. Algorithms that follow the cul-de-sac topology excel; those that cut across it fragment walking paths.

TABLE I
Five Diverse Test Neighborhoods

Hood	Addrs	Segs	Dens. (/km^2)	Grid (%)	Character
PAC	145	11	9686	9	Dense commercial
FM	307	43	796	28	Cul-de-sacs
CC	489	117	736	71	Regular grid
RP	584	289	70	44	Sparse campus
SG	268	44	100	20	Elongated

Community Center (CC). 489 addresses on 117 segments in a regular grid (71% of streets within 25° of cardinal directions). All algorithms perform reasonably because the grid aligns with both chain ordering and Voronoi boundaries.

Research Park (RP). 584 addresses spread across 289 segments at only 70 addresses/km^2. The sparse campus layout means most walking is unproductive regardless of algorithm. This is the "universal failure case."

Southgate (SG). 268 addresses on 44 segments in an elongated layout (aspect ratio 1.9). The single dominant axis aligns with Euclidean Voronoi boundaries, making this the neighborhood where Voronoi wins.

V. Algorithms

All algorithms operate on the segment graph: segments are nodes with four ports and precomputed cost matrices; edges carry road-network distances between shared intersections. We describe eleven algorithms below, each with pseudocode.

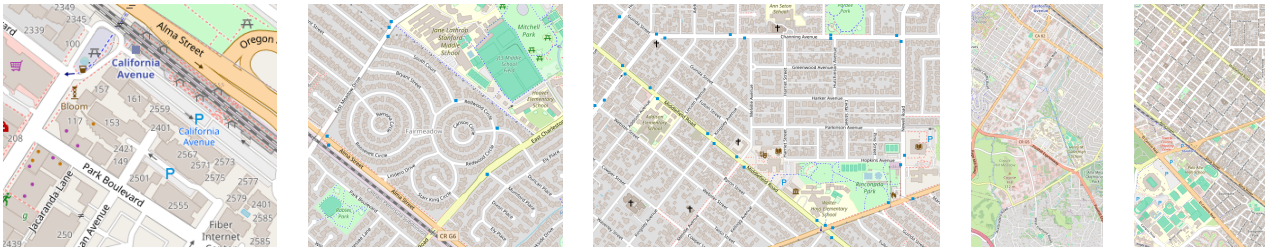
A. Chain+Slice (Nearest-Neighbor)

Build a single-worker walk order using nearest-neighbor on ports: from the current exit port, select the unvisited segment minimizing transition + traversal cost. Partition the chain into N groups at address-count boundaries $\lceil A/N \rceil$. Try all starting segments; keep the best.

Complexity: $O(M^2 \cdot 16)$ per starting segment (for each of M segments, evaluate 16 port pairs against all remaining), times M starting segments. Total: $O(M^3)$, mitigated by the precomputed distance matrix which makes each port-pair lookup $O(1)$.

Algorithm 1 Chain+Slice

Require: Segments $\mathcal{S}$ with cost matrices, worker count N
best $\leftarrow \infty$
for each starting segment $s_0 \in \mathcal{S}$ **do**
  chain $\leftarrow [s_0]$; mark s_0 visited
  while unvisited segments remain **do**
    $s^* \leftarrow \arg\min_{s \notin \text{chain}} [\text{transition}(\text{chain.last}, s) + C_s]$
    Append s^* to chain
  end while
  Slice chain into N groups at $\lceil A/N \rceil$ boundaries
  Score partition; update best if improved
end for



PAC (145a, 11s) **FM** (307a, 43s) **CC** (489a, 117s) **RP** (584a, 289s) **SG** (268a, 44s)

Fig. 3. OpenStreetMap views of the five test neighborhoods. PAC is a dense commercial cluster around California Avenue; FM has radial cul-de-sacs (Starr King Circle visible at center); CC is a regular grid; RP is a sparse campus with many roads but few addresses; SG is elongated along a single axis. These five layouts produce the widest spread of algorithm performance in our study. Map data © OpenStreetMap contributors.

Chain+Slice on Fairmeadow (106/107/99)

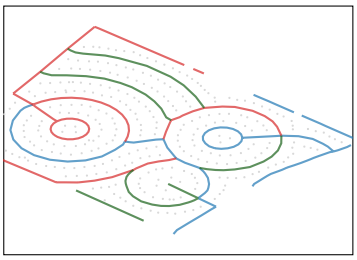


Fig. 4. Chain+Slice (NN) on Fairmeadow, $N = 3$. Three ESWs' segments are colored blue, red, green. The chain follows a nearest-neighbor path; cut points at 106/107/99 addresses produce geographically interleaved partitions.

B. Right-Hand Rule Chain

A variant of Chain+Slice that adds a directional bias. After exiting a segment, candidates are scored by:

$$\text{score}(s) = d_{\text{road}}(\text{exit}, \text{entry}_s) + 0.5 \cdot \theta_{\text{cw}} + 0.1 \cdot C_s \qquad (12)$$

where d_{road} is the road-network transition distance, $\theta_{\text{cw}} \in [0°, 360°)$ is the clockwise turn angle from the current heading to the candidate entry point, and C_s is the candidate's traversal cost. The 0.5 weight on turn angle (in degrees, so $0.5 \times 180° = 90\,\text{m}$ equivalent for a U-turn) biases toward continuing in roughly the same direction, producing a spiral traversal pattern. The 0.1 weight on segment cost is a tiebreaker favoring cheaper segments.

Complexity: Same as NN, $O(M^3)$.

Algorithm 2 Right-Hand Rule Chain

Require: Segments $\mathcal{S}$, worker count N, current heading $\vec{h}$
best $\leftarrow \infty$
for each starting segment $s_0 \in \mathcal{S}$ **do**
  chain $\leftarrow [s_0]$; $\vec{h} \leftarrow$ direction of s_0
  while unvisited segments remain **do**
    $s^* \leftarrow \arg\min_s [d_{\text{road}} + 0.5 \cdot \theta_{\text{cw}}(\vec{h}, s) + 0.1 \cdot C_s]$
    Append s^*; $\vec{h} \leftarrow$ exit direction of s^*
  end while
  Slice into N groups; score; update best
end for

Fig. 5 compares NN and RH chain traversals on Fairmeadow.

Nearest-Neighbor Chain **Right-Hand Rule Chain**

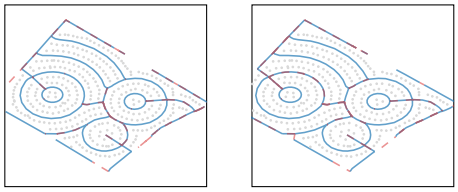


Fig. 5. Fairmeadow single-worker traversal (43 segments). Blue = segment traversals (productive). Red dashed = inter-segment transitions (unproductive). Left: NN greedily selects the nearest unvisited segment, occasionally jumping across the neighborhood. Right: RH rule's clockwise bias produces a tighter spiral with shorter transitions.

Right-Hand Rule on Fairmeadow (116/117/79)

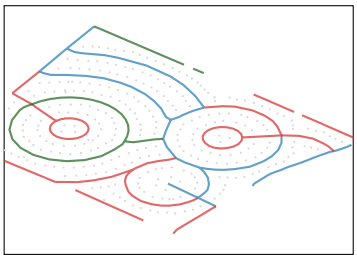


Fig. 6. Right-Hand Rule on Fairmeadow, $N = 3$ (116/117/79). The spiral pattern produces more geographically contiguous partitions than NN, but the third ESW (green) gets fewer addresses because the spiral exhausts the interior first.

C. 2-Opt Local Search

Starting from the best NN chain, iteratively improve segment ordering by reversing sub-chains. For each pair (i, j) in the chain, reverse the sub-chain $[i..j]$ if the reversal reduces total walk distance. When a sub-chain is reversed, each segment's traversal direction flips: the entry and exit ports swap, and the segment cost is re-read from the cost matrix with transposed indices. Transition costs between the reversed segments and their neighbors are recomputed via the road-network distance matrix. Iterate until no improving swap exists (or 500 passes).

Why 2-opt helps: The NN chain is greedy — it selects the next segment by minimizing immediate cost without considering downstream consequences. 2-opt repairs these local mistakes by finding pairs of transitions that form a "cross" (where reversing the intermediate sub-chain "uncrosses" them, reducing total distance). On our data, 2-opt reduces worst-case walk by 3.2% on average and up to 41.3% in the best case.

Complexity: $O(M^2)$ per pass, typically converging in $O(M)$ passes, giving $O(M^3)$ total.

Algorithm 3 2-Opt Local Search

Require: Chain from best Chain+Slice run
repeat
  improved ← false
  for each pair (i, j) in chain **do**
    Reverse sub-chain $[i..j]$; swap entry/exit ports
    Recompute transition costs at boundaries
    if total walk distance decreased **then**
      Accept reversal; improved ← true
    else
      Undo reversal
    end if
  end for
until ¬improved or 500 passes
Slice into N groups; score

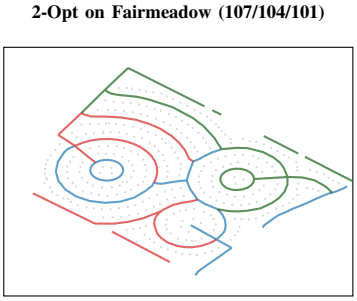


Fig. 7. 2-Opt on Fairmeadow, $N = 3$ (107/104/101). Local search "uncrosses" the NN chain's transitions, producing a well-balanced partition (60% walk quality) with visibly more contiguous regions than raw Chain+Slice.

D. Recursive Bisection

Split segments by centroid along the longer geographic axis (latitude or longitude) at the address-count midpoint. Recurse until N groups remain. Within each group, build an NN chain to order segments. Produces compact rectangular regions with good geographic contiguity, but axis-aligned cuts may not follow street patterns.

Complexity: $O(M \log N)$ for splitting, plus $O((M/N)^3)$ for NN within each group.

E. Voronoi with Balanced Seeds

Assign segments to the nearest of N seed points by Euclidean distance (consistent with straight-line Voronoi partition boundaries). Segments straddling a cell boundary (endpoints in different cells) are reassigned wholly to the closer cell.

Algorithm 4 Recursive Bisection

Require: Segments $\mathcal{S}$, worker count N
groups ← $[\mathcal{S}]$
while $|\text{groups}| < N$ **do**
  Select largest group G
  axis ← longer of latitude, longitude span of G
  Sort G by centroid along axis
  Split G at address-count midpoint into G_1, G_2
  Replace G with G_1, G_2 in groups
end while
for each group G_k **do**
  Order segments in G_k by NN chain
end for

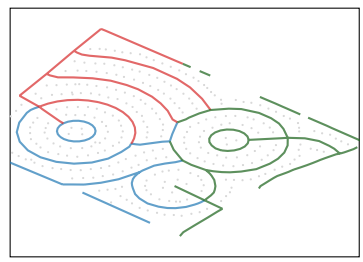


Fig. 8. Recursive Bisection on Fairmeadow, $N = 3$ (109/102/101). Axis-aligned cuts produce compact rectangular regions. Each cell is geographically contiguous by construction, but the straight cut lines may not follow street patterns.

Seed optimization. Standard Lloyd's iteration [8] minimizes within-cell variance, not address count balance — it converges to unbalanced cells (Fig. 9, left). We replace it with greedy hill-climbing: starting from Lloyd's result, try moving each seed to random road nodes (50 trials per seed, 10 restarts), keeping any move that reduces imbalance. For small neighborhoods (≤ 41 nodes, $N = 3$), we use exhaustive search over all $\binom{|V|}{N}$ seed combinations (Fig. 9, right). Within each cell, segments are ordered by NN chain.

F. Voronoi + Boundary Transfer (Vor+BT)

Starting from the Voronoi balanced-seeds result, iteratively transfer boundary segments from the heaviest cell (by time) to a lighter adjacent cell, provided the transfer does not disconnect the source cell. Connectivity is checked by BFS on the segment adjacency graph. Transfers continue until no improving move exceeds a 0.5-minute threshold or 200 iterations are reached. Within each resulting cell, segments are ordered by NN chain.

The key limitation: Voronoi cells in road networks tend to be *tree-shaped* — removing any interior segment disconnects the cell. Only cells with cycles (redundant connections) allow productive transfers. This means Vor+BT helps most on grid neighborhoods and least on cul-de-sac or sparse layouts.

Fig. 10 shows the boundary transfer process on Midtown, a large Palo Alto grid neighborhood (5,140 addresses, 581 segments) included here to illustrate boundary effects at scale.

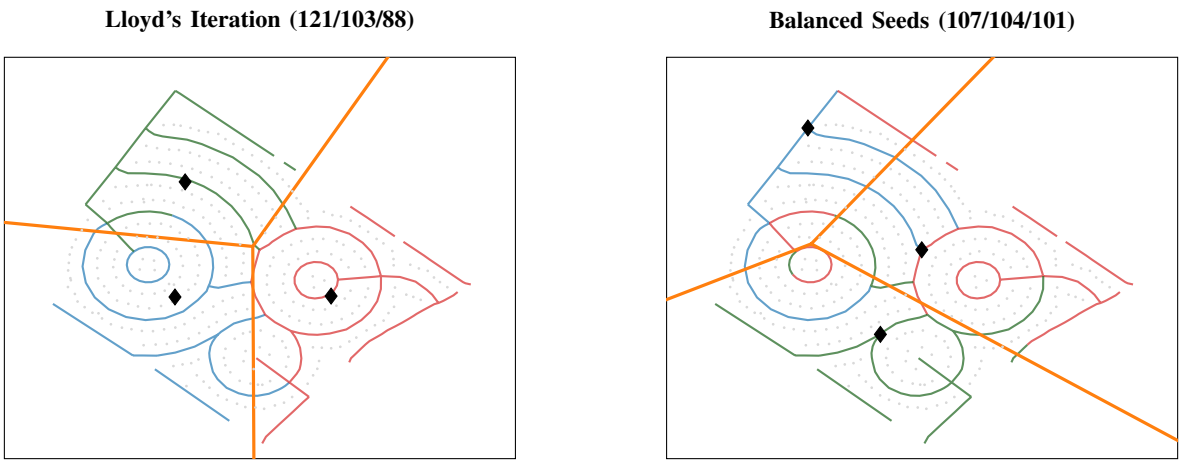


Fig. 9. Fairmeadow Voronoi ($N = 3$). Left: Lloyd's iteration converges to unbalanced cells (121/103/88) because it minimizes variance, not balance. Right: exhaustive seed search achieves near-perfect balance (107/104/101) with 53% walk quality. Orange = Voronoi partition lines. Black diamonds = seeds. Same framework, different seed optimization — seed placement is the critical factor.

Algorithm 5 Voronoi with Balanced Seeds

Require: Segments $\mathcal{S}$, worker count N, road nodes V
Initialize seeds via Lloyd's iteration (address-weighted centroids)
for 10 random restarts **do**
  for each seed $k = 1 \dots N$ **do**
    for 50 random trials **do**
      Move seed k to random node $v \in V$
      Assign segments to nearest seed (Euclidean)
      if address-count imbalance decreased **then**
        Keep new seed position
      end if
    end for
  end for
end for
Reassign straddling segments wholly to closer cell
Order each cell's segments by NN chain

The straight Voronoi partition lines (left) cut through 14 segments that must be reassigned wholly to one cell (right).

G. BFS (Breadth-First Search)

Assign segments round-robin: each round, for each ESW k, assign the unvisited segment minimizing marginal time increase:

$$\Delta T_k = \frac{d_{\text{road}}(\text{frontier}_k, \text{entry}_s) + C(\text{in}_s, \text{out}_s)}{v} + n_s \cdot t_{\text{assess}} \tag{13}$$

where frontier_k is ESW k's current exit port and n_s is the segment's address count. Produces "pizza slice" partitions radiating from the huddle, but these interleave geographically, doubling unproductive walking vs. chain methods.

H. DFS (Depth-First Search)

Build one ESW's assignment at a time. Compute a target finish time $T^* = (\text{total walk time} + A \cdot t_{\text{assess}})/N$. For ESW 1, greedily add nearest segments until $T_1 \approx T^*$. Repeat for ESW 2 from the huddle with remaining segments. The last ESW takes all remaining segments. Suffers from the "dregs problem": the last ESW inherits scattered remnants, producing time spreads up to 391 min on sparse neighborhoods.

Algorithm 6 Voronoi + Boundary Transfer

Require: Voronoi partition from balanced seeds
for iter $= 1$ to 200 **do**
  $k_{\max} \leftarrow$ cell with highest estimated time
  for each boundary segment s in cell $k_{\max}$ **do**
    if removing s would disconnect $k_{\max}$ **then**
      Skip s
    end if
    for each adjacent cell k_{adj} **do**
      Compute improvement from transferring s
    end for
  end for
  if best improvement < 0.5 min **then**
    break
  end if
  Transfer best segment; update cell membership
end for
Order each cell's segments by NN chain

Algorithm 7 BFS

Require: Segments $\mathcal{S}$, worker count N
$\text{frontier}_k \leftarrow$ huddle point for all $k = 1 \dots N$
while unassigned segments remain **do**
  for $k = 1$ to N **do**
    $s^* \leftarrow \arg\min_s \Delta T_k(s)$ {Eq. 13}
    Assign s^* to ESW k; update frontier_k
  end for
end while

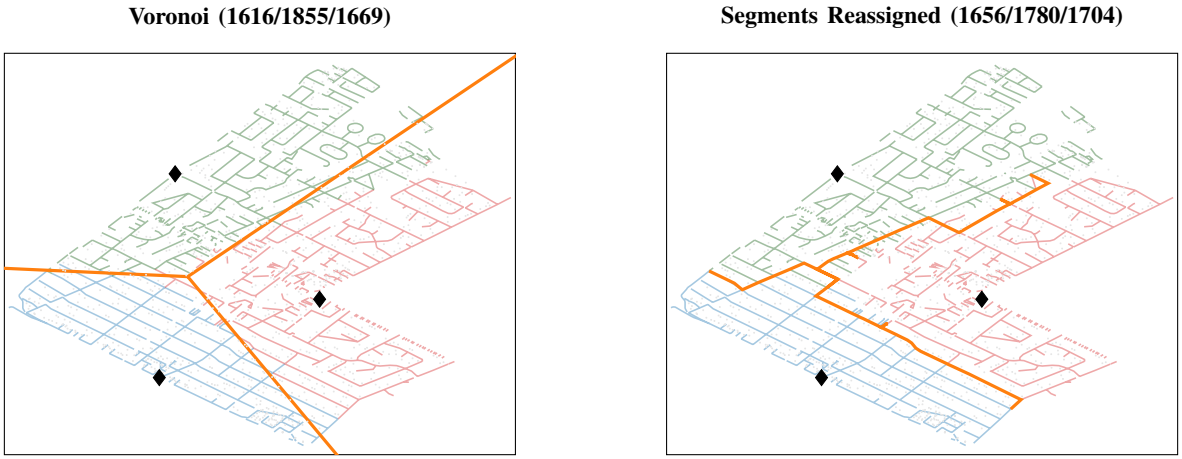


Fig. 10. Midtown ($N = 3$, 5,140 addresses). Left: Voronoi partition with orange straight-line cell boundaries. These geometric boundaries cut through 14 road segments, splitting streets between cells. Right: each split segment is reassigned wholly to one cell (by closer endpoint). The cell boundary now follows segment edges rather than straight lines. Black diamonds = Voronoi seeds.

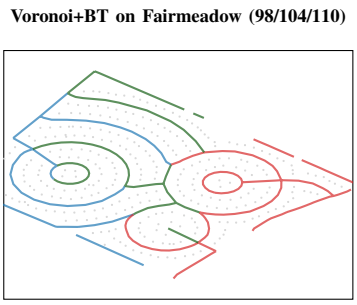


Fig. 11. Voronoi+BT on Fairmeadow, $N = 3$ (98/104/110). Boundary transfers adjust the Voronoi partition to improve time balance. Each cell is geographically contiguous; segments within each cell are ordered by NN chain.

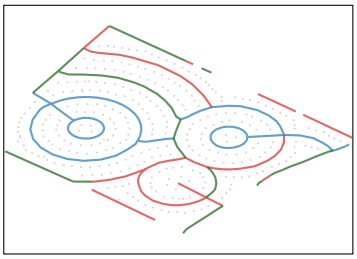


Fig. 12. BFS on Fairmeadow, $N = 3$ (141/103/68). Round-robin assignment from the huddle interleaves territories. ESW 1 (blue) grabs 141 addresses while ESW 3 (green) gets only 68 — the "pizza slice" pattern creates severe count imbalance.

Algorithm 8 DFS

Require: Segments $\mathcal{S}$, worker count N
$T^* \leftarrow (\text{total walk} + A \cdot t_{\text{assess}})/N$
for $k = 1$ to $N - 1$ **do**
  frontier $\leftarrow$ huddle point; $T_k \leftarrow 0$
  while $T_k < T^*$ and unassigned segments remain **do**
    $s^* \leftarrow$ nearest unassigned segment to frontier
    Assign s^* to ESW k; update frontier and T_k
  end while
end for
Assign all remaining segments to ESW N

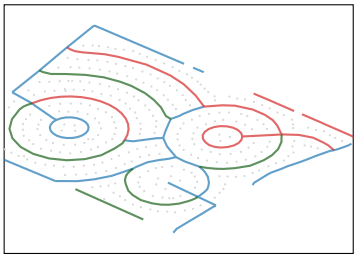


Fig. 13. DFS on Fairmeadow, $N = 3$ (105/102/105). Sequential assignment produces well-balanced counts on this neighborhood, but the last ESW (green) may inherit scattered remnants on other neighborhoods (the "dregs problem").

I. SA Post-Processing

Starting from BFS's result, apply simulated annealing [14]: transfer boundary segments from the heaviest ESW to the lightest, accepting worse solutions with Boltzmann probability $e^{-\Delta/T}$ to escape local optima. Temperature cools from $T_0 = 0.5 \cdot \text{spread}$ to $T_f = 0.01$ over 500 iterations. The key mechanism: a two-move backtracking sequence can achieve gains that no single greedy transfer finds (12.5% improvement demonstrated on Greater Miranda — see Section VII-G1).

J. Oracle

Since no single strategy dominates, the oracle runs all strategies and selects the best by score (Eq. 14).

Runtime. For M segments and I intersection nodes: Dijkstra precomputation is $O(I \cdot (I + E) \log I)$ where E is edge count (one-time, 2–41 s in practice). The oracle body is $O(M^3)$ for each chain strategy (dominated by M starting

Algorithm 9 SA Post-Processing

Require: Partition from BFS, temperature schedule $T_0 \to T_f$

for iter $= 1$ to 500 **do**
  $k_{\max} \leftarrow \arg\max_k T_k$; select boundary segment s
  $k_{\text{target}} \leftarrow$ adjacent ESW with lowest T_k
  Transfer s: $k_{\max} \to k_{\text{target}}$
  $\Delta \leftarrow \max_k T_k^{\text{new}} - \max_k T_k^{\text{old}}$
  if $\Delta < 0$ or rand() $< e^{-\Delta/T}$ **then**
    Accept transfer
  else
    Reject; undo transfer
  end if
  Cool: $T \leftarrow T \cdot \alpha$
end for

Algorithm 10 Oracle

Require: Segments $\mathcal{S}$, worker count N

Run Chain+Slice, Right-Hand, 2-Opt, Bisect, Voronoi, Vor+BT, BFS, DFS, BFS+SA
Score each by $2 \cdot W_{\max} + W_{\text{total}} + 5000 \cdot \text{imbalance}$
return partition with lowest score

segments $\times$ M^2 chain construction). For neighborhoods up to 289 segments, the oracle completes in <1 s after precomputation.

Score vs. walk quality. The scoring function (Eq. 14) minimizes worst-case walking distance and imbalance, not walk quality directly. This means the oracle may select a partition with lower $W_{\max}$ but worse walk quality — for example, on Southgate the oracle selects a chain method (28% WQ) over Voronoi+BT (59% WQ) because the chain method produces a shorter worst-case walk. Section VIII discusses the implications of this tension.

K. Scoring Function

Candidate assignments are scored by:

$$\text{Score} = 2 \cdot W_{\max} + W_{\text{total}} + 5000 \cdot \frac{\sum_k ||P_k| - A/N|}{A/N} \quad (14)$$

The three terms serve distinct purposes:

- $2 \cdot W_{\max}$: **Fairness.** The worst-case ESW walk, weighted $2\times$ because the objective (Eq. 7) is to minimize the maximum.
- W_{total}: **Efficiency.** Total distance across all ESWs. Two assignments with the same $W_{\max}$ are distinguished by total effort.
- $5000 \cdot \text{imbalance}$: **Balance penalty.** The constant 5000 was chosen empirically so that a 10% address-count imbalance adds 500 m equivalent — comparable to one inter-segment transition. This prevents grossly unbalanced partitions while allowing minor imbalance if it reduces walking.

L. Random Baseline

Shuffle segments, partition by address count, NN chain within each group. Run 30 trials, keep best. Quantifies the value of optimization: the oracle improves over random by 25% on average.

Algorithm 11 Random Baseline

Require: Segments $\mathcal{S}$, worker count N, trials $R = 30$

best $\leftarrow \infty$
for $r = 1$ to R **do**
  Randomly shuffle $\mathcal{S}$
  Partition into N groups at $\lceil A/N \rceil$ address boundaries
  Order each group by NN chain
  Score partition; update best if improved
end for

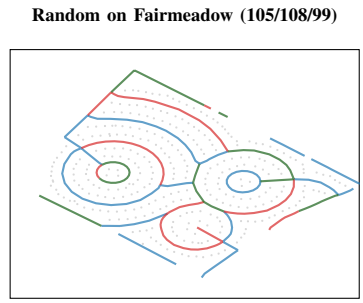


Fig. 14. Random assignment on Fairmeadow, $N = 3$ (105/108/99). Despite balanced counts (random shuffle + count slicing), the geographic interleaving is extreme — segments of all three colors are scattered throughout. Walk quality: 43%.

VI. Simulation Methodology

Data pipeline. OpenStreetMap data is imported via `osm2pgsql` from a Geofabrik California PBF (1.3 GB), filtered to the Palo Alto bounding box. Road segments are split at intersection nodes. Municipal addresses are snapped to segments by perpendicular projection. Cost matrices are precomputed from address positions and street widths (estimated from OSM road class).

Distance computation. Dijkstra shortest paths are precomputed between all intersection nodes within each neighborhood (2–31 s depending on size). All distances — inter-segment transitions, huddle-to-first-segment walks, and algorithm evaluations — use actual walkable road-network paths.

Parameters. As specified in the general constraints (Section II): $t_{\text{assess}} = 5$ min/address, $v = 5$ km/h, huddle at nearest intersection to serving fire station.

Test matrix. 11 algorithms $\times$ 5 neighborhoods $\times$ $N \in \{3, 5, 7, 10\}$ = 220 test cases. Each evaluated with all three metrics. All algorithms use the same segment data, distance matrices, huddle points, and evaluation function. The walk from huddle to first segment is included in every algorithm's unproductive distance.

VII. Results

A. Master Comparison ($N = 3$)

Table II shows walk quality for all algorithms on all neighborhoods at $N = 3$. The productive distance row confirms

the invariant: within each neighborhood, it is approximately constant regardless of algorithm.

TABLE II
WALK QUALITY (%) BY ALGORITHM AND NEIGHBORHOOD ($N = 3$). PRODUCTIVE DISTANCE IS INVARIANT (SHOWN ONCE PER NEIGHBORHOOD). BEST PER NEIGHBORHOOD IN BOLD. ADDRESS COUNTS SHOWN ARE PER-NEIGHBORHOOD; ALGORITHM FIGURES MAY REPORT SLIGHTLY HIGHER TOTALS BECAUSE BOUNDARY-STRADDLING SEGMENTS INCLUDE ADDRESSES FROM ADJACENT NEIGHBORHOODS.

Algorithm	**PAC** 145a	**FM** 307a	**CC** 489a	**RP** 584a	**SG** 268a
Prod. (km)	*0.7*	*6.7*	*11.8*	*17.1*	*5.4*
ChainNN	**59**	61	62	23	28
ChainRH	56	58	61	**26**	27
2-Opt	**59**	60	**66**	25	28
Bisect	29	56	63	24	28
Voronoi	29	53	64	24	**56**
Vor+BT	29	57	61	24	**59**
BFS	35	62	60	20	24
DFS	53	**66**	65	**26**	24
BFS+SA	34	64	60	21	22
Random	25	43	43	15	24
Oracle	56	58	62	**26**	28

TABLE III
TIME SPREAD (MINUTES) BY ALGORITHM AND NEIGHBORHOOD ($N = 3$). LOWER IS BETTER. BEST PER NEIGHBORHOOD IN BOLD.

Algorithm	**PAC** 145a	**FM** 307a	**CC** 489a	**RP** 584a	**SG** 268a
ChainNN	55	43	**10**	128	24
ChainRH	52	**6**	12	95	25
2-Opt	55	23	12	138	**18**
Bisect	167	39	32	330	20
Voronoi	**35**	42	37	368	109
Vor+BT	**35**	49	32	471	108
BFS	255	364	96	628	739
DFS	51	32	173	391	60
BFS+SA	102	55	95	**3**	31
Random	54	7	11	105	45
Oracle	52	**6**	**10**	95	**18**

Key findings:

- **Productive distance is approximately constant** within each neighborhood, confirming the invariant (Eq. 11). Variation is $\leq$5% on dense neighborhoods; on sparse layouts like Research Park, port-choice differences widen the tolerance to $\sim$10%.
- **DFS and 2-opt achieve highest walk quality** on most neighborhoods (DFS: FM 66%, CC 65%; 2-opt: PAC 59%, CC 66%).
- **Voronoi+BT wins on Southgate** (59%) by 31 points over chain methods (28%). The elongated shape aligns naturally with Euclidean partitioning; boundary transfers further improve balance.
- **Research Park defeats all algorithms**: best is 26% (ChainRH, DFS). At 70 addresses/km^2, most walking is unproductive regardless of strategy.
- **BFS is consistently among the worst**: 20–62% walk quality (only the random baseline is worse). Round-robin fan-out interleaves territories.
- **Oracle improvement over random**: the oracle achieves 26–62% walk quality vs. 15–43% for random.

B. Scaling with Team Size

TABLE IV
ORACLE WINNER BY TEAM SIZE AND NEIGHBORHOOD

Neighborhood	N=3	N=5	N=7	N=10
PAC	ChainRH	ChainRH	ChainRH	ChainRH
FM	ChainRH	ChainRH	ChainNN	TwoOpt
CC	ChainNN	ChainNN	ChainNN	TwoOpt
RP	ChainRH	ChainRH	TwoOpt	TwoOpt
SG	TwoOpt	TwoOpt	ChainRH	TwoOpt

The oracle winner shifts with N: ChainRH dominates at small N on most neighborhoods, while TwoOpt takes over at larger N as intra-partition ordering becomes more important than partition boundaries.

C. Walk Quality Visualization

Fig. 15 shows the productive-distance invariant visually: productive distance is constant at 6.7 km for all algorithms on Fairmeadow; only unproductive distance varies.

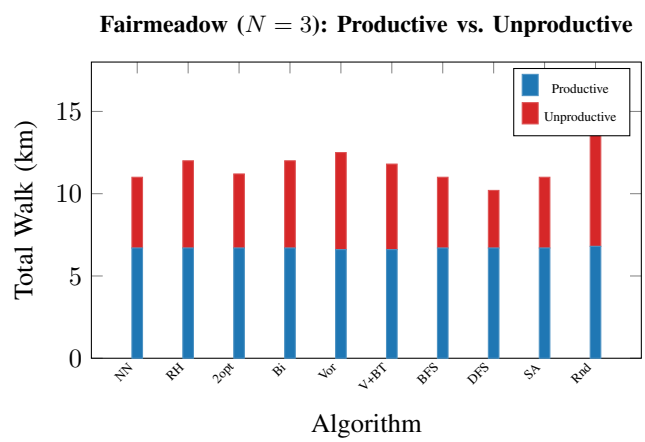


Fig. 15. Fairmeadow ($N = 3$): productive (blue, constant at 6.7 km) vs. unproductive (red, varies 3.5–9.2 km). The productive-distance invariant is confirmed: all algorithms visit the same segments. DFS achieves the lowest unproductive distance; Random the highest.

D. Time Spread

Fig. 17 shows time spread for key algorithms at $N = 3$. BFS produces extreme spreads (96–739 min) because its round-robin pattern creates unbalanced address counts. Chain methods maintain spreads at or below 55 min on most neighborhoods; Voronoi+BT has moderate spreads (32–471 min) that vary with neighborhood geometry.

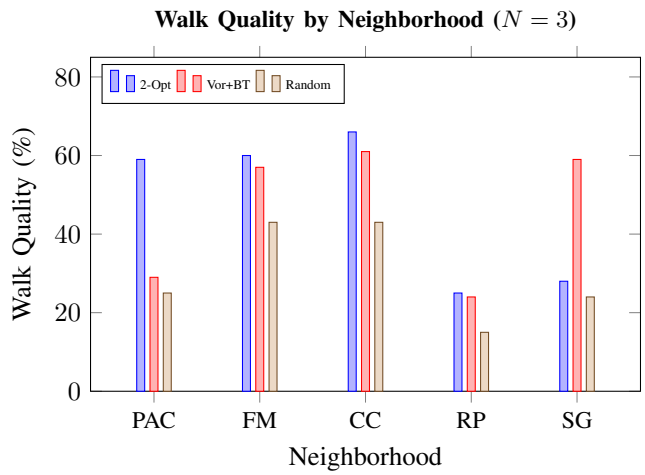


Fig. 16. Walk quality for 2-opt, Voronoi+BT, and Random baseline across 5 neighborhoods ($N = 3$). 2-opt leads on compact and grid neighborhoods but Voronoi+BT wins on Southgate (59% vs. 28%). Random quantifies the cost of not optimizing (15–43%).

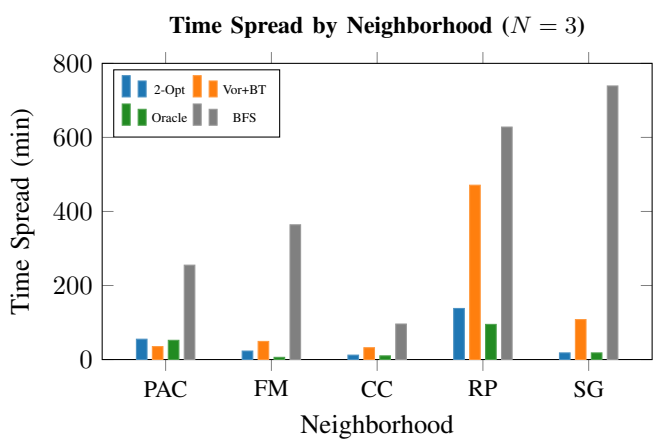


Fig. 17. Time spread (max − min ESW finish time) at $N = 3$. Blue = 2-Opt, orange = Vor+BT, green = Oracle, gray = BFS. BFS produces extreme spreads due to address-count imbalance. The oracle achieves low spreads on all neighborhoods except Research Park (sparse campus where all methods struggle).

E. Walking Trail Visualizations

Figs. 18 and 19 show per-ESW walking trails for two contrasting neighborhoods. Fig. 18 uses Fairmeadow (one of the five test neighborhoods). Fig. 19 uses Crescent Park, a larger Palo Alto neighborhood (1,476 addresses, 256 segments, regular grid), included here to illustrate trail behavior at scale on a rectilinear layout.

F. Overhead Decomposition

The overhead ratio ρ has two sources: (1) unproductive walking (transitions), and (2) productive imbalance (some ESWs traverse more segment distance than others). We can decompose:

$$\rho = \frac{W_{\max}}{W^*} = \frac{W_{\max}^{\text{prod}} + W_{\max}^{\text{unprod}}}{W^*} \tag{15}$$

For Fairmeadow ($\rho = 1.90$): $W^* = 2{,}187\,\text{m}$, $W_{\max} = 4{,}162\,\text{m}$ (DFS, including huddle walk). The worst-case ESW has $\sim$2,400 m productive and $\sim$1,762 m unproductive (of which the huddle walk is a significant component). If all transitions including the huddle walk could be eliminated, ρ would drop to $\sim$1.10 — the remaining overhead is from productive imbalance (some ESWs get slightly longer segments).

Implication: The path to reducing ρ is primarily through smarter transition routing (e.g., solving a TSP on segment endpoints), not through better partitioning.

G. Time-Based Analysis

We also evaluate all algorithms under the time metric (Eq. 8), with all workers starting from shared huddle points. The key findings:

Assessment time dominates. At $t_{\text{assess}} = 5\,\text{min}$, assessment time is $\sim 8\times$ walking time for typical neighborhoods. The time metric is therefore dominated by address count balance, which chain+slice handles naturally through its $\lceil A/N \rceil$ slicing.

BFS produces operationally unacceptable paths. While BFS's total walking distance is comparable to chain methods when the huddle walk is included (11.0 km vs. 10.2 km on Fairmeadow), the real problem is its catastrophic time spread (364 min on Fairmeadow) and geographic interleaving. The round-robin fan-out pattern assigns addresses to ESWs in a "pizza slice" pattern that forces extensive backtracking. From an operational standpoint, ESWs walking past unvisited addresses to reach their assigned territory would find this unacceptable regardless of what the time metric shows. This reveals that **walking quality** must be treated as a constraint, not just a component of the objective.

DFS produces the best of both worlds on compact neighborhoods: balanced addresses (105/102/105 on Fairmeadow), the highest walk quality (66%), and the least total walking (10.2 km). But it suffers from the "dregs problem" on irregular neighborhoods where the last ESW inherits scattered remnants.

Fig. 20 shows the DFS assignment for Fairmeadow under the time metric.

1) Worked Example: SA on Greater Miranda: We trace simulated annealing on Greater Miranda, a small Palo Alto neighborhood (54 addresses, 14 segments) chosen because its compact size makes the SA trajectory easy to follow ($N = 3$). Starting from BFS's result (ESW times: 106/142/75 min, max = 142 min), SA searches for boundary transfers that reduce the maximum. Table V shows the critical two-move sequence.

Final result: ESW 1 = 124 min (21 addrs), ESW 2 = 124 min (21 addrs), ESW 3 = 75 min (12 addrs). The two heaviest ESWs are now perfectly balanced. The key insight: SA's ability to accept temporarily worsening moves enables two-step improvements that no greedy search can find.

H. Voronoi Analysis

Voronoi with balanced seeds achieves 24–64% walk quality at $N = 3$; adding boundary transfers (Voronoi+BT) improves this to 29–61% on most neighborhoods and 59% on Southgate.

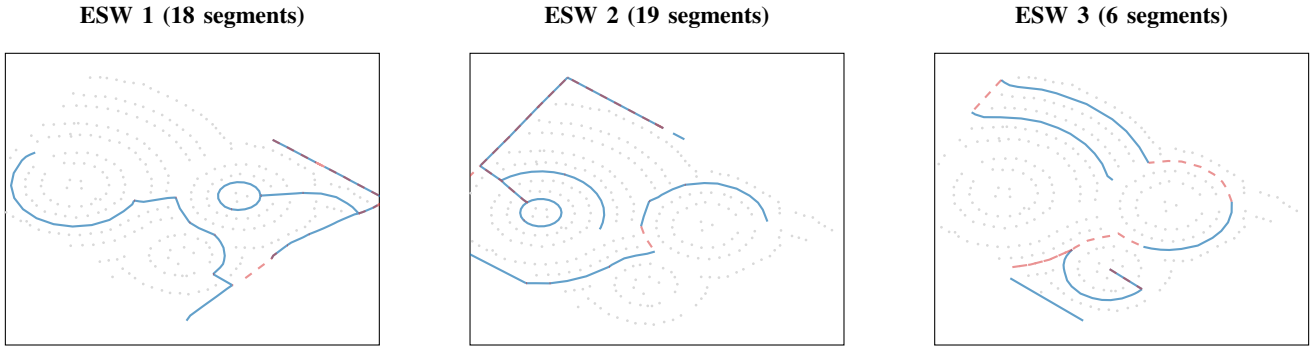


Fig. 18. Fairmeadow (307 addresses, $N = 3$, 2-opt result). Each panel is one ESW's route. Blue = productive walking within road segments. Red dashed = unproductive transitions following the road network. Gray dots = all addresses. Fairmeadow's compact cul-de-sac layout keeps transitions short. ESW 3 has only 6 segments, reflecting a naturally isolated cul-de-sac cluster.

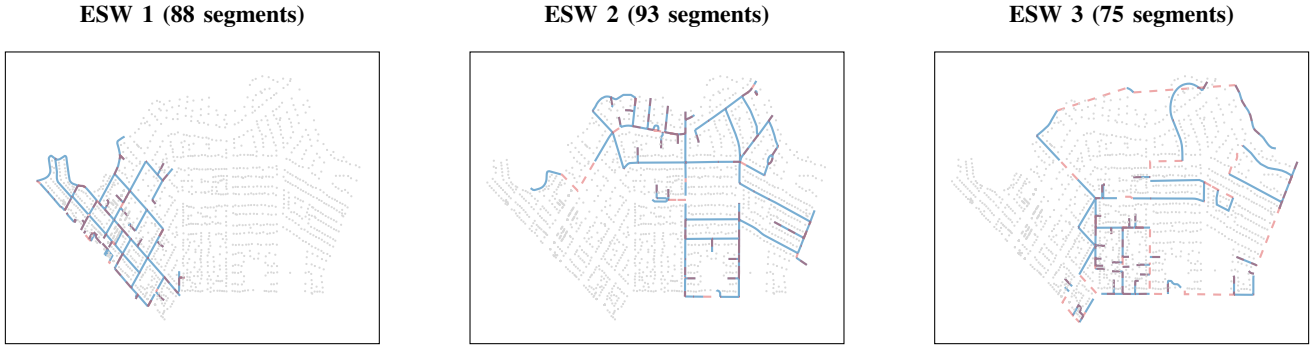


Fig. 19. Crescent Park (1,476 addresses, $N = 3$, 2-opt result). The 2-opt-improved chain produces geographically contiguous assignments on this rectilinear grid. Red transition paths follow actual streets through intersections — not straight-line jumps — reflecting road-network-constrained routing.

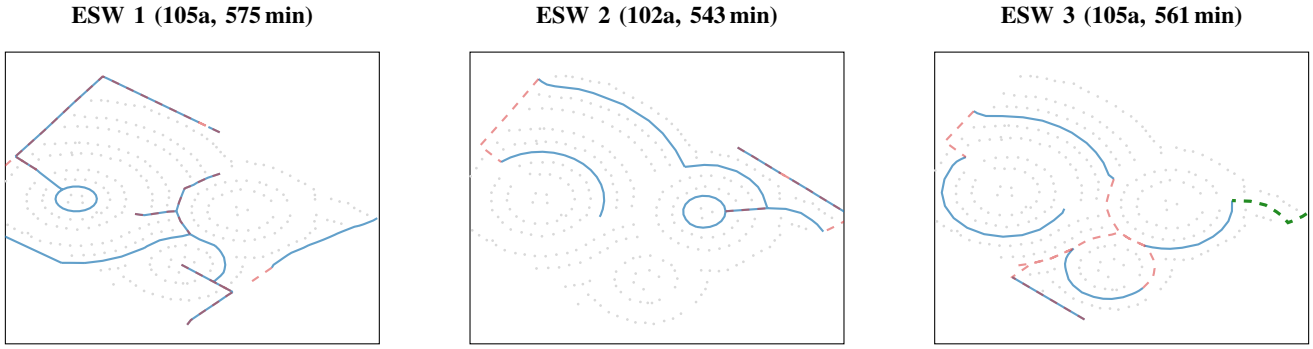


Fig. 20. Fairmeadow under DFS ($N = 3$, time metric). Green diamond = huddle point (fan-out from Fire Station 4). Green dashed = walk from huddle to first segment. Blue = productive. Red dashed = unproductive transitions. DFS produces well-balanced address counts (105/102/105) with 10.2 km total walking and the highest walk quality (66%) of any algorithm on this neighborhood.

The comparison with naive Lloyd's iteration remains dramatic: seed placement matters far more than the assignment metric.

Two factors determine whether Voronoi methods succeed:

Neighborhood shape. Voronoi+BT's advantage is most visible on Southgate (59% vs. 28% for chain methods) where the Euclidean partition aligns with the elongated shape. The straight Voronoi boundary naturally bisects the long axis, assigning each ESW a compact longitudinal strip. Chain methods must snake through the elongated layout, producing interleaved partitions with high unproductive walking. On compact neighborhoods (PAC), this advantage reverses: any Voronoi boundary cuts through the dense cluster, creating 89% more unproductive walking than chain methods.

Network topology. On grids (CC), boundary transfers achieve 24 min spread at $N = 7$ — the best of any algorithm — because grid cells have redundant connections that allow productive segment transfers. On tree-shaped networks (cul-de-sacs, sparse campus), transfers have little effect because removing any segment disconnects the cell. Fairmeadow's radial cul-de-sac layout falls between these extremes: Voronoi+BT

TABLE V

SA BACKTRACKING TRACE ON GREATER MIRANDA. AFTER EARLY ITERATIONS REDUCE THE MAXIMUM FROM 142 TO 124 MIN, SA ACCEPTS A *worsening* MOVE (MAX INCREASES FROM 124 TO 135) BECAUSE THE BOLTZMANN PROBABILITY $e^{-\Delta/T}$ IS HIGH. THIS ENABLES THE NEXT KEY MOVE: TRANSFERRING MOANA CT FROM THE NOW-HEAVY ESW 3 TO THE NOW-LIGHT ESW 2 DROPS THE MAXIMUM BACK TO 124 MIN. THE TWO-MOVE SEQUENCE ACHIEVES A 12.5% IMPROVEMENT OVER THE BFS STARTING POINT THAT NO SINGLE GREEDY TRANSFER COULD FIND.

Step	ESW Times Before	After	Δmax
BFS start		[106, 142, 75]	max = 142
After early iterations	[106, 142, 75]	[124, 68, 124]	–18
Miranda Grn (14a): ESW2→ESW1	[124, 68, 124]	[124, 64, 135]	+11 (accepted)
Moana Ct (11a): ESW3→ESW2	[124, 64, 135]	[**124, 124**, 75]	**–11**
Final		[124, 124, 75]	max = **124** (–12.5%)

achieves 57% walk quality (vs. DFS's 66%), competitive but not dominant.

I. Constraint Tension Analysis

We formulate all discovered constraints simultaneously and attempt to solve within them:

Hard constraints:

1) **Walkable contiguity**: Each ESW's segments form a walk-ordered chain.
2) **Walking quality floor**: $Q_k \geq Q_{\min} = 50\%$ for each ESW.
3) **Full coverage**: Every segment assigned to exactly one ESW.

Objective: Minimize finish-time spread subject to constraints 1–3.

Result: only 9% of algorithm-cases are feasible. Across 5 neighborhoods at $N = 3, 5, 7, 10$ (220 algorithm-cases), only 19 achieve $Q_{\min} \geq 50\%$ for the worst ESW. Only 4 of 20 test configurations (neighborhood $\times$ N) have *any* feasible algorithm — 3 on Community Center (the regular grid) and 1 on Fairmeadow.

The infeasibility is structural. The median walk quality of the worst ESW is only 20%. The dominant factor is the **walk from the huddle point to each ESW's first assigned segment**, which is unavoidable unproductive overhead.

The fundamental tension: Walkable contiguity + time balance is achievable (chain+slice produces 70 min average spread). Walking quality $\geq 50\%$ requires that most walking is productive, which is difficult when the huddle-to-first-segment distance is large relative to the intra-neighborhood walking.

VIII. DISCUSSION

A. When Each Algorithm Wins

The five diverse neighborhoods partition the algorithm landscape:

- **Dense, compact (PAC):** Chain methods dominate (59% WQ). All segments are adjacent; transitions are inherently short. Voronoi and Bisect struggle because any partition boundary creates unnecessary crossings in this small cluster.
- **Cul-de-sacs (FM):** DFS wins (66%) by building contiguous territories from the huddle. Voronoi+BT is competitive (57%) with boundary transfers closing the gap.
- **Regular grid (CC):** 2-opt and DFS lead (66%, 65%). The grid aligns with both chain ordering and Voronoi boundaries; all methods perform within 60–66%.
- **Sparse campus (RP):** Everything fails (<26% WQ). The fundamental problem is not the algorithm but the data: 289 segments with only 584 addresses means 2.0 addresses per segment — most walking is between segments with no addresses.
- **Elongated (SG):** Voronoi+BT wins decisively (59% vs. 28% for chain methods). The Euclidean partition naturally bisects the elongated shape; boundary transfers further improve balance. Chain methods snake through the length, producing interleaved partitions.

B. Cross-Study Comparison

Table VI compares the oracle and Voronoi on the 5 diverse neighborhoods. Productive walking is nearly identical — the same addresses must be visited. The differentiator is *unproductive walking*, which varies dramatically with neighborhood character.

Key findings:

- **Oracle dominates on compact, dense neighborhoods** (PAC: 56% vs. 29%). Chain ordering keeps transitions short when segments are close together. Voronoi+BT's geometric partition cuts through the dense cluster, creating 89% more unproductive walking on PAC.
- **The gap nearly vanishes on cul-de-sacs and grids** (FM: 58% vs. 57%; CC: 62% vs. 61%). Voronoi boundaries align well with these topologies; boundary transfers further close the gap.
- **Both methods struggle on sparse layouts** (RP: 26% vs. 24%). When addresses are spread thinly over many segments, no partitioning can avoid long transitions.
- **Voronoi+BT wins decisively on elongated neighborhoods** (SG: 59% vs. 28%, –37% unproductive). The Euclidean partition naturally bisects the elongated shape, while chain+slice must snake through it.

Fig. 21 shows the productive/unproductive breakdown visually.

TABLE VI

CROSS-STUDY COMPARISON ($N = 3$). PRODUCTIVE DISTANCE IS CONSTANT; UNPRODUCTIVE VARIES BY METHOD AND NEIGHBORHOOD. ΔUNPROD = HOW MUCH MORE UNPRODUCTIVE WALKING VORONOI+BT PRODUCES VS. THE ORACLE (NEGATIVE = VORONOI+BT IS BETTER).

			Oracle			**Voronoi+BT**			
Neighborhood	**Addrs**	**Character**	Prod (km)	Unprod (km)	WQ (%)	Prod (km)	Unprod (km)	WQ (%)	ΔUnprod
Palo Alto Central	145	Dense commercial	0.7	0.9	56	0.7	1.7	29	+89%
Fairmeadow	307	Cul-de-sacs	6.7	5.3	58	6.6	5.2	57	–2%
Community Center	489	Regular grid	11.8	7.4	62	11.9	7.8	61	+5%
Research Park	584	Sparse campus	19.0	58.2	26	17.4	59.8	24	+3%
Southgate	268	Elongated	5.3	13.2	28	5.4	8.3	59	**–37%**

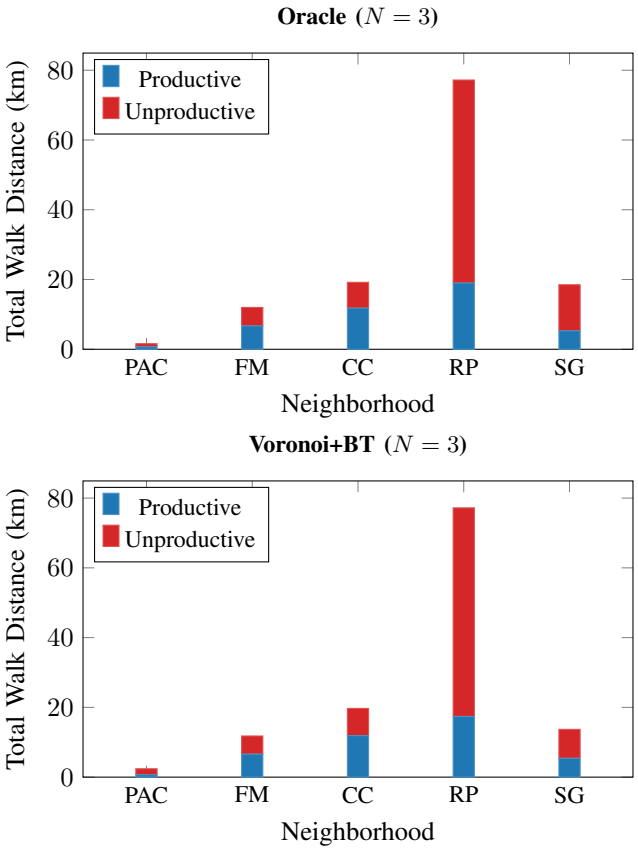


Fig. 21. Productive vs. unproductive walking for oracle (top) and Voronoi+BT (bottom) on 5 neighborhoods at $N = 3$. Blue = productive, red = unproductive in both panels. Productive bars are nearly identical; unproductive bars tell the story. Note Research Park's extreme unproductive walking (both >70% unproductive) vs. Fairmeadow where both methods achieve ∼58% productive.

C. The Productive-Distance Invariant

Table II confirms that productive distance is approximately invariant across all algorithms within each neighborhood (≤5% variation on dense neighborhoods; up to ∼10% on sparse layouts where port-choice effects are amplified). This is the paper's deepest insight: **algorithms cannot reduce work — they can only reduce waste.** The practical implication: when evaluating a new algorithm, compare its *unproductive* distance to the best known, not its total distance.

D. Score vs. Walk Quality

The scoring function (Eq. 14) optimizes worst-case walking distance and address-count balance, not walk quality directly. This produces a concrete tension: on Southgate, the oracle selects a chain method (28% walk quality, $W_{\max} = 13.5$ km) over Voronoi+BT (59% walk quality, $W_{\max} = 14.8$ km). The chain method is "better" by $W_{\max}$, but the Voronoi+BT assignment is operationally superior — ESWs spend more time assessing and less time walking past unvisited addresses.

This tension arises because walk quality and worst-case distance measure different things. Low $W_{\max}$ means the slowest ESW finishes sooner; high walk quality means ESWs perceive their assignments as sensible. When the two conflict, operational experience suggests walk quality should take precedence (Insight 1), because ESWs who lose confidence in their assignments may deviate from prescribed routes. Incorporating walk quality into the scoring function — or using it as a hard constraint — is a direction for future work.

E. Walk-Ordered Sequences vs. Geographic Partitions

On compact, cul-de-sac, and grid neighborhoods, algorithms that build assignments as walk-ordered sequences of segments (chain+slice, 2-opt, DFS) outperform those that partition geographic space first (BFS, Voronoi, bisect at small N). The reason: walking is inherently sequential — an ESW visits one address, then the next, along a street. Algorithms that respect this sequential structure produce lower unproductive walking because consecutive segments in the chain are geographically adjacent. The exception is elongated neighborhoods, where geographic partitioning (Voronoi+BT) produces superior walk quality because the Euclidean partition aligns with the dominant axis (Section VII-H).

F. Recommended Practice

The analyses converge on a concrete operational procedure:

1) **Run the oracle.** The oracle runs all strategies — including chain methods and Voronoi+BT — and selects the best by score in under one second.
2) **Check walk quality.** If the worst ESW's walk quality is below 50%, inspect the neighborhood geometry. On elongated layouts, Voronoi+BT typically produces higher walk quality than chain methods (Section VII-H). The oracle's score function optimizes distance, not walk

quality, so manually selecting the Voronoi+BT result may be warranted when walk quality is the priority.

3) **Check finish-time spread.** Compute T_k for each ESW (Eq. 8) with the NPC's estimated t_{assess}. If the spread exceeds the NPC's tolerance (e.g., 30 minutes), manually transfer one or two boundary segments from the slowest ESW to the fastest.
4) **Brief ESWs using the trail maps.** Show each ESW their walking trail (blue = productive segments, red = transitions). ESWs who can see that their route is logical and efficient — mostly blue, minimal red — will work with confidence rather than frustration.

This procedure addresses the key constraints: walkable contiguity (chain+slice), time balance (oracle scoring + manual adjustment), walking quality (step 2), and ESW morale (trail visualization).

G. Limitations

Single city. All neighborhoods are in Palo Alto — an affluent suburb with regular grids, cul-de-sacs, and flat terrain. Results may differ in dense urban, rural, or irregular-street environments.

No field validation. ESWs have not yet been observed using these assignments. A structured field trial is the most important next step.

Equal service time. All addresses are treated equally. Multi-unit buildings and high-rises require different assessment times, which the time metric could accommodate with per-address weights.

Flat terrain. Walking speed is assumed constant (5 km/h). In hilly environments, a time-based metric with elevation-adjusted speed would be needed.

IX. Related Work

The Capacitated Vehicle Routing Problem (CVRP) [3] distributes service locations among vehicles from a depot. Our problem shares the depot concept (all workers depart from a shared huddle point), but differs in requiring address-count balance, walking-speed traversal, and street-constrained routing.

Postal delivery route optimization [6] is the most closely analogous domain: letter carriers walk along street segments visiting addresses on both sides, with route design that must balance workload and minimize deadheading (our "unproductive walking"). Our cost matrix formulation is equivalent to the carrier's side-of-street decision at each block, and our sub-segment analysis (Section II-B4) formalizes when assigning opposite sides independently could reduce total walking. Census enumeration area delineation [7] similarly partitions addresses into balanced geographic groups, though without the sequential ordering requirement.

The within-segment traversal problem — visiting addresses on both sides of a street — relates to the Chinese Postman Problem [15], which seeks the shortest closed walk traversing every edge of a graph. Our formulation differs in that only address-bearing portions of each edge require traversal, and the entry/exit ports are degrees of freedom. The within-segment problem is also a special case of the TSP on parallel lines [11].

Voronoi diagrams [16] provide a natural spatial partitioning where each cell contains the points closest to one seed. Our application uses Euclidean Voronoi for segment assignment with balanced seed search, followed by contiguity-preserving boundary transfers (Section V-F). The key finding — that seed placement matters far more than the assignment metric — echoes results in facility location [8], where Lloyd's iteration minimizes variance but not balance.

K-means [8] and balanced variants [9] cluster geographically but ignore street connectivity. The nearest-neighbor TSP heuristic [10] produces $O(\log n)$-competitive tours. Recursive bisection [12], [13] is standard for balanced spatial decomposition. Our application to road-network segments with port-aware traversal costs appears novel. Simulated annealing [14] provides the post-optimization framework.

X. Conclusion and Future Work

We presented a practical system for assigning addresses to emergency services workers that minimizes worst-case walking distance using road-network-constrained routing. We summarize the evidence for each contribution:

C1 (Segment model). Decomposing the OSM road network into 64,482 segments with geometric address snapping (41,854 addresses, median snap distance 22.9 m, 99% within 50 m) provides a physically accurate model of how ESWs walk. Fig. 1 verifies geometric correctness of the side-of-street classification.

C2 (Cost matrix). The 4×4 matrix captures zigzag, U-turn, through-walk, and out-and-back in a unified formulation. 88% of both-side segments favor zigzag; the 12% favoring U-turn are identified automatically by the $\bar{g}/w$ ratio. Port choice produces 74% cost variation on a single segment (Section II-B5), justifying the matrix.

C3 (Three metrics). Walk quality, time spread, and the productive-distance invariant capture what matters operationally. The invariant reveals that **66% of walking is unproductive on average** (including the huddle-to-first-segment walk), confirming that transition reduction is the dominant optimization target.

C4 (Empirical evaluation). Across 5 diverse neighborhoods at $N = 3, 5, 7, 10$ (220 test cases), no single algorithm dominates. The winning strategy depends on neighborhood geometry: DFS/2-opt on compact/grid (59–66%), Voronoi+BT on elongated (59%), all fail on sparse (<26%).

C5 (Oracle algorithm). Running all strategies completes in <1 s for neighborhoods up to 289 segments, enabling real-time use during emergency operations.

C6 (Time-based analysis). When assessment time dominates walking ($\sim$8×), chain+slice still produces the best assignments because its contiguous geographic partitions naturally minimize unproductive walking. BFS produces twice the walking of DFS/chain+slice, revealing that walking quality must be a constraint.

C7 (Voronoi with balanced seeds). Balanced seed search transforms Voronoi from a failure (Lloyd's produces unbalanced cells) into a competitive approach. Adding boundary transfers (Voronoi+BT) wins decisively on elongated neighborhoods (59% on Southgate vs. 28% for chain methods). Seed placement matters far more than the assignment metric.

C8 (Constraint tension). Walkable contiguity + time balance + walking quality $\geq$ 50% is feasible in only 9% of algorithm-cases. The walk from the huddle to the first assigned segment dominates walking quality (median worst-ESW quality 20%).

Cross-study insight. The productive-distance invariant reveals that algorithms can only reduce waste, not work. This reframes future research: the objective is not "better routing" but "less unproductive walking." Promising directions include Voronoi + chain+slice hybrids (Voronoi for geographic partitioning, chain+slice for intra-cell routing), constraint-solver formulations that explicitly balance walk quality and time spread, and field validation with ESW teams.

Future work includes: adaptive $Q_{\min}$ based on huddle-to-neighborhood distance; field validation with ESW teams; and testing on cities with different street geometries.

References

[1] Federal Emergency Management Agency, "Damage Assessment Operations Manual," FEMA P-2198, 2023.

[2] B. Iannucci, "ES-Link: Mobile Situational Awareness for Emergency Services," Iannucci Wireless Systems Engineering, Palo Alto, CA, 2025.

[3] P. Toth and D. Vigo, *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. Philadelphia, PA: SIAM, 2014.

[4] K. Andreev and H. Räcke, "Balanced graph partitioning," *Theory of Computing Systems*, vol. 39, no. 6, pp. 929–939, 2006.

[5] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W. H. Freeman, 1979.

[6] H. A. Eiselt, M. Gendreau, and G. Laporte, "Arc routing problems, part II: The rural postman problem," *Operations Research*, vol. 43, no. 3, pp. 399–414, 1995.

[7] P. J. Taylor, "Enumeration area design for census data collection," *Journal of the Royal Statistical Society: Series A*, vol. 175, no. 3, pp. 669–686, 2012.

[8] S. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inform. Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[9] M. I. Malinen and P. Fränti, "Balanced k-means for clustering," in *Proc. Joint IAPR Int. Workshops Structural and Syntactic Pattern Recognition*, 2014, pp. 32–41.

[10] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis, "An analysis of several heuristics for the traveling salesman problem," *SIAM J. Comput.*, vol. 6, no. 3, pp. 563–581, 1977.

[11] G. N. Frederickson, M. S. Hecht, and C. E. Kim, "Approximation algorithms for some routing problems," *SIAM J. Comput.*, vol. 7, no. 2, pp. 178–193, 1978.

[12] M. J. Berger and S. H. Bokhari, "A partitioning strategy for nonuniform problems on multiprocessors," *IEEE Trans. Comput.*, vol. C-36, no. 5, pp. 570–580, 1987.

[13] G. Karypis and V. Kumar, "Multilevel algorithms for multi-constraint graph partitioning," in *Proc. ACM/IEEE Conf. Supercomputing*, 1998.

[14] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[15] J. Edmonds and E. L. Johnson, "Matching, Euler tours and the Chinese postman," *Math. Programming*, vol. 5, no. 1, pp. 88–124, 1973.

[16] F. Aurenhammer, "Voronoi diagrams — a survey of a fundamental geometric data structure," *ACM Comput. Surv.*, vol. 23, no. 3, pp. 345–405, 1991.